HomeMesh: A Local-First Wall-Mounted Smart Home Control Panel Using an ESP32 and the Home Assistant WebSocket API

Garima Nidhi
24BCE1782
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

Taniska Routray
24BCE1578
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

Bharadhwaj
24BCE1959
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

Pranesh S
24BCE5138
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India

***Abstract*—Commercial smart home wall panels are expensive and often depend on vendor cloud services, while low-cost microcontroller panels usually reach only the devices that speak one protocol. This paper presents HomeMesh, a wall-mounted control panel built on an ESP32-WROOM-32 with a 3.2-inch 320×240 capacitive touchscreen, which controls any device known to a local Home Assistant server through its WebSocket API. Because the full state dump of even a small installation exceeds the memory available on the microcontroller, the work is split: a desktop companion tool performs entity discovery and authors the panel layout, which is delivered to the panel as a Home Assistant event and stored in non-volatile memory, while the firmware subscribes only to the entities in its layout. On a test installation with 123 entities, the complete state dump measured 57,671 bytes, whereas the panel's subscription delivered the initial state of its seven entities in a single 2,403-byte message. The firmware runs networking and the LVGL user interface on separate processor cores, keeps every parse bounded, and leaves 15.6 KB of its 40 KB graphics memory pool free. We also report a field diagnosis in which intermittent link loss was traced to the host laptop's single-radio Wi-Fi adapter moving the hotspot to the 5 GHz band, together with the firmware change that made the panel resilient to such faults.**

Index Terms*—ESP32, Home Assistant, Internet of Things, LVGL, smart home, WebSocket, embedded user interface, local-first systems

I. Introduction

Smart home devices are now commonplace, and most are controlled through phone applications or voice assistants that route commands through vendor cloud services. This dependence adds latency, fails when the Internet connection fails, and sends a continuous record of household activity off the premises; processing at the edge, inside the home, is widely argued to reduce all three [1], [2]. A fixed wall panel complements these interfaces: it gives every occupant, including visitors and people without a smartphone, immediate control of the room they are standing in [3].

A panel is only useful if it can reach the devices actually installed, and smart home devices remain fragmented across Zigbee, Z-Wave, Wi-Fi and proprietary cloud ecosystems. Interoperability has been the central problem of home automation for two decades [4], and the Matter standard is the most recent attempt to address it [5]. Home Assistant, an open-source home automation server, takes a pragmatic route: it integrates devices from many ecosystems and exposes them through a single, uniform API. A panel that talks to Home Assistant rather than to individual devices inherits that reach.

Existing ESP32-based smart home systems typically switch relays and read sensors attached to the board itself, and are controlled through a web page or a cloud dashboard [6]–[8]. Panels built on a message broker such as MQTT, the dominant IoT messaging protocol [9]–[12], see only devices that publish themselves to that broker. Comparative studies report that WebSocket gives the lowest latency for interactive, request-driven exchanges, while MQTT is more economical for streaming telemetry [13]–[16]. A control panel is interactive by nature, which favours a WebSocket design.

The obstacle is memory. The ESP32-WROOM-32 has no external RAM, and the complete state of a real Home Assistant installation is far larger than the microcontroller can parse. This paper makes the following contributions:

1) an architecture that divides the work between a desktop tool and the microcontroller so that every message the panel handles is small and bounded;
2) a runtime layout mechanism in which the panel's pages, devices, icons and colours are authored on a PC and delivered over the panel's existing connection, with no reflashing;
3) a dual-core firmware design for the ESP32 with measured memory, flash and stack usage; and
4) an evaluation of message size and latency, and a field case study of a network fault and its root cause.

Section II describes the methodology, Section III presents and discusses the results, and Section IV concludes.

II. Methodology

A. System Architecture

Fig. 1 shows the system. A Home Assistant server runs in a Docker container on a laptop, which also hosts the 2.4 GHz Wi-Fi network that the panel joins; no cloud service is involved. A desktop companion tool, the layout designer, runs on the same laptop. The panel is a self-contained ESP32 board with a touchscreen.

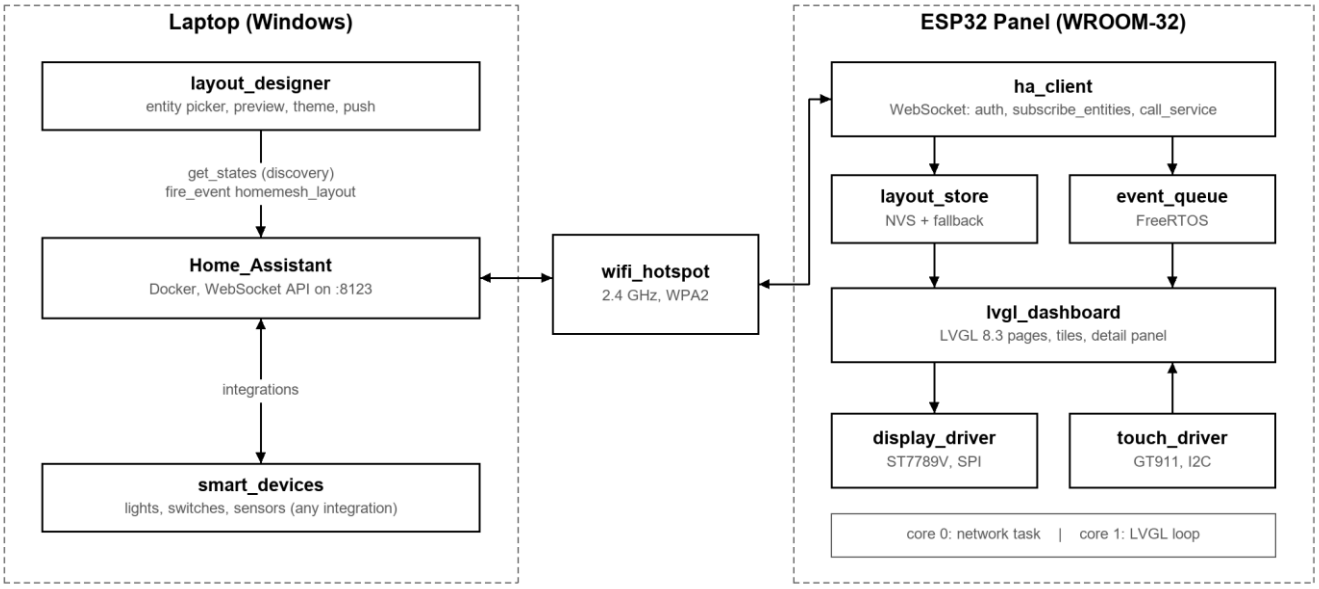


Fig. 1. System block diagram. The laptop hosts Home Assistant, the layout designer and the 2.4 GHz hotspot; the panel holds one WebSocket connection to Home Assistant. Devices shown under the laptop are reached through Home Assistant integrations.

The division of work follows from one measurement. Home Assistant's *get_states* command returns the full state of every entity; on our test installation of 123 entities it returned 57,671 bytes, an average of 469 bytes per entity. That alone exceeds the panel's entire 40 KB graphics memory pool, and a household with several hundred entities would produce hundreds of kilobytes. The panel therefore never calls *get_states*. The desktop tool performs discovery and produces a layout naming a handful of specific entities; the panel subscribes to exactly those entities and to nothing else.

B. Hardware Platform

The panel is an ESP32-2432S032 board: an ESP32-WROOM-32 module (dual-core Xtensa LX6 at 240 MHz, 520 KB of on-chip SRAM, 4 MB of flash, no PSRAM), a 3.2-inch 240×320 IPS display driven by an ST7789V controller over SPI at 27 MHz, a GT911 capacitive touch controller on I²C, and a CH340 USB-to-serial bridge. The interface runs in landscape at 320×240. Because this board family ships with varying display and touch controllers, both were identified empirically, and all hardware-specific code is confined to two driver modules so that the user interface and network code are independent of the batch. The ESP32 radio is 2.4 GHz only, which constrains the network (Section III-D). Table I lists the components.

TABLE I
PANEL HARDWARE

Component	Part	Interface
Microcontroller	ESP32-WROOM-32, 2×240 MHz, no PSRAM	—
Display	3.2" IPS 240×320, ST7789V	SPI, 27 MHz
Touch	GT911 capacitive	I²C
USB serial	CH340	UART
Storage	4 MB flash, 3 MB app, NVS	SPI flash
Expansion	microSD slot (unused)	SPI

C. Communication with Home Assistant

The panel holds a single WebSocket connection to Home Assistant. After authenticating with a long-lived access token it sends two subscriptions: *subscribe_entities*, listing the entity identifiers in its layout, and *subscribe_events* for the layout-delivery event described in Section II-D. Home Assistant answers the first with one snapshot message containing the current state of every listed entity, then sends only the attributes that change. Devices are controlled with *call_service*, whose form is identical for every integration, so control logic does not depend on how a device is connected. A disconnected socket is the availability signal: the interface greys out until the connection is re-established, and a 15 s heartbeat detects a silently dropped route. Fig. 2 shows the resulting message sequence.

Table II compares the three designs considered for this link. The panel was first built on MQTT with simulated devices, which limited it to entities published to a broker. The second design used Home Assistant's REST interface to fetch each entity's initial state and the WebSocket for updates; it reached every entity, but its requests blocked the connection (Section III-D). The final design takes both the initial state and all updates from one subscription.

TABLE II
PANEL-TO-SERVER DESIGNS CONSIDERED

Design	Devices reached	Initial state	Main drawback
MQTT broker (first)	Only those publishing to the broker	Retained messages	Most HA devices invisible
REST + WebSocket (second)	All HA entities	One HTTP request per entity	Requests block the socket
WebSocket subscription (final)	All HA entities	One snapshot message	Snapshot bounded by 15 KB frame

Every incoming frame is classified on its raw bytes before parsing. Control messages, such as authentication results and command acknowledgements, pass through a small ArduinoJson filter into a fixed 768-byte document. State snapshots and layout deliveries, which are larger, are routed to a dedicated 4 KB document whose filter keeps only state, brightness and colour; a wildcard key matches any entity identifier. Fixed documents drop keys that do not fit without raising an error, so overflow is checked and reported explicitly. A change message omits the state field when only an attribute changed, such as the brightness of a lamp that remains on, so state is treated as optional when updates are applied. Listing 1 summarises the classification.

Listing 1. Frame classification in the network task.

```
on_text_frame(frame):
  if frame contains '"homemesh_layout"':
    parse_layout(frame); validate; store; restart
  elif frame contains '"event":{"a":'  (snapshot)
    or frame contains '"event":{"c":'  (change):
    for each entity in frame:
      if state present: post POWER event
      if brightness/colour present: post event
  else:  (control message, small filter)
    auth_ok -> subscribe; result -> log failure
```

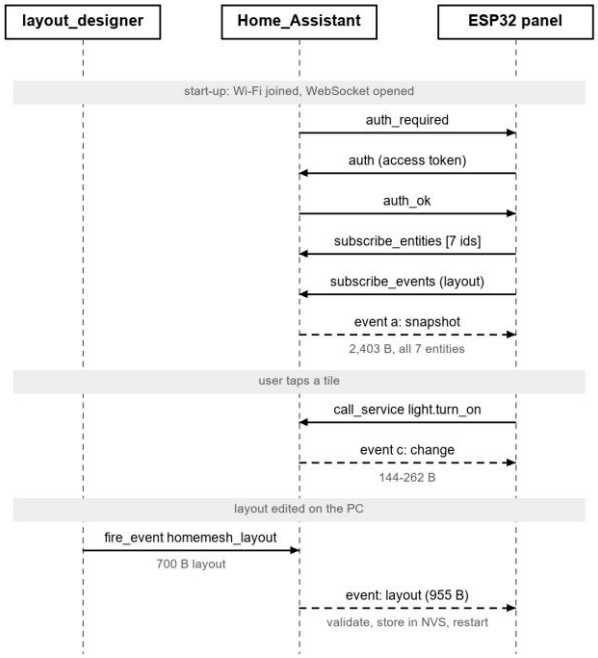


Fig. 2. Message sequence between the layout designer, Home Assistant and the panel, with measured message sizes. Dashed arrows are messages Home Assistant pushes to the panel.

D. Layout as Data

The layout designer is a Python application that uses only the standard tkinter toolkit, so it runs on any PC without a build toolchain. It calls *get_states*, groups entities by room, and lets the user choose which to show, mark favourites for the Home page, reorder tiles by dragging, pick an icon per device and adjust nine theme colours. A preview draws each 320×240 page using the same geometry as the firmware (Fig. 3). Entities that have no room in Home Assistant are listed but cannot be placed, since the panel's pages are rooms.

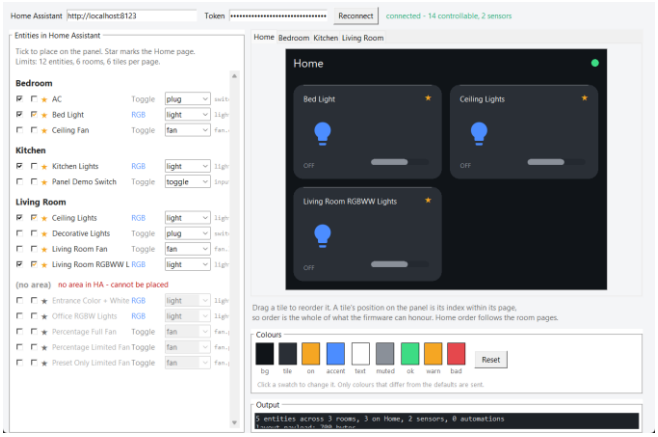


Fig. 3. The layout designer connected to Home Assistant: entities grouped by room with placement, favourite and icon controls (left), a live preview of the selected panel page (right), and the nine theme colours.

The result is a compact JSON document listing rooms, entities (identifier, name, kind, room, favourite flag and icon), sensors and theme; Listing 2 shows an excerpt. Rather than running a server on the panel, the designer fires a custom Home Assistant event carrying the layout; the 700-byte test layout arrives as a 955-byte event. The panel receives it on its existing connection, so neither side needs to know the panel's IP address. The firmware validates the whole layout before storing anything: strings are copied into a fixed 2,560-byte arena so a layout cannot grow into memory the interface needs, and a rejected layout leaves the running one untouched and reports the reason on the serial console. An accepted layout is written to non-volatile storage (NVS) and the board restarts; rebuilding the whole interface in place would fragment the graphics memory pool, whereas a restart takes a few seconds and is deterministic. A layout compiled into the firmware serves as a fallback when NVS is empty.

Listing 2. Excerpt of a layout document.

```
{"v":1,
 "areas":["Bedroom","Kitchen","Living Room"],
 "entities":[
  {"id":"light.bed_light","name":"Bed Light",
   "kind":"RGB","area":0,"fav":true,
   "icon":"light"}, ...],
 "sensors":[
  {"id":"sensor.outside_temperature",
   "area":2,"hum":false}, ...]}
```

E. Firmware Organisation

The firmware uses the Arduino framework on FreeRTOS. A network task pinned to core 0 runs Wi-Fi, the WebSocket client and message parsing; it runs at priority 0, since at priority 1 it starved the idle task and tripped the watchdog. The LVGL 8.3 interface runs on core 1 and is serviced every 5 ms. The two communicate only through a FreeRTOS queue of small event records, so the interface never waits on the network. LVGL allocates from a fixed 40 KB pool. Large JSON documents are placed in static memory rather than on the 12 KB network task stack: an early design that declared a 4 KB document locally overflowed the stack, because the compiler inlined it into the per-frame handler. The firmware

also reports its own state: LVGL assertion failures are logged with file, line and pool usage, and Wi-Fi failures are logged with the driver's reason code.

F. User Interface

The interface has a header showing the page title, the room's temperature and humidity, and a link indicator. Pages are swiped horizontally: a Home page of favourites and one page per room, each holding up to six tiles in a two-column grid (Fig. 4). A tile shows the device name, an icon, its state and, for dimmable lights, a brightness bar; a chevron opens a detail panel with an on/off control, a brightness slider and a colour control. Icons are 16 Material Design glyphs compiled into LVGL fonts at 28 and 18 pixels; drawn like text from flash, they cost about 7.6 KB of flash and nothing from the graphics pool, unlike decoded bitmap images. When a device is on, its tile lifts slightly, its icon turns amber inside a tinted disc, and its reading and brightness bar turn amber; a dimmed light shows its level, such as 72%. Text widths were checked against the font's glyph metrics so that the widest reading, 100% at 51 pixels, fits beside the brightness bar. Fills are flat: gradients were tried and removed, because the 16-bit RGB565 display has only 32 levels of red and blue, and a subtle ramp on a dark colour showed visible, tinted bands.

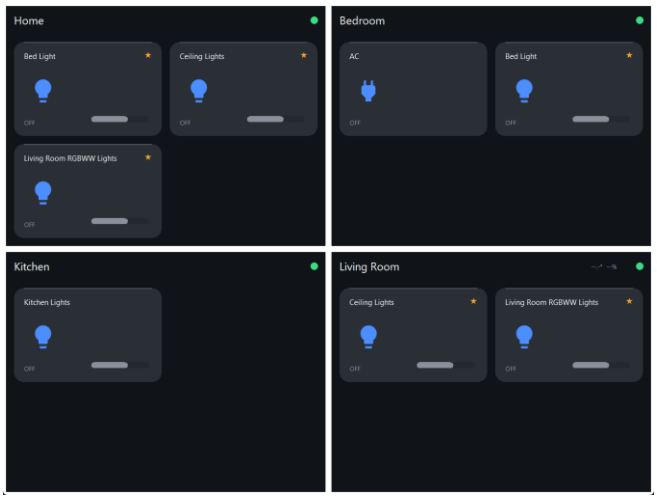


Fig. 4. The panel's Home page and three room pages for the test layout, as drawn by the layout designer's preview with every device off.

III. Results and Discussion

A. Test Setup

Home Assistant 2026.9 ran in Docker Desktop on a Windows 11 laptop, with its demonstration integration providing lights, switches, fans and sensors (123 entities in total). The panel's layout held five controllable entities and two sensors across three rooms. The network was a Windows Mobile Hotspot on 2.4 GHz with WPA2. Server-side figures were measured by a Python WebSocket client on the laptop; panel-side figures come from the panel's serial log and from PlatformIO's build report.

B. Message Size and Latency

TABLE III
Message Size and Latency

Quantity	Value
Full state dump, get_states (123 entities)	57,671 B
Average size per entity	469 B
Panel snapshot, subscribe_entities (7 entities)	2,403 B
Change message: brightness only / light off	144 B / 262 B
Layout delivery event (700 B layout)	955 B
Snapshot delivered after subscribing (server)	11.5 ms
Service call to state-change message, median of 10 (server)	3.2 ms
Panel round-trip time over hotspot, mean of 6	40 ms, 0% loss

Table III and Fig. 5 summarise the measurements. The panel's initial-state snapshot was 2,403 bytes for its seven entities, 24 times smaller than the full state dump, and, more importantly, its size depends on the layout rather than on the size of the installation. Subsequent changes were 144 to 262 bytes. Home Assistant delivered the snapshot 11.5 ms after the subscription request, and a service call produced the corresponding state-change message after a median of 3.2 ms over ten trials. These server-side times exclude the Wi-Fi hop; the panel's round-trip time over the hotspot averaged 40 ms with no loss. Both are well below the threshold at which a touch response feels delayed. The results agree with reports that WebSocket offers low latency for interactive exchanges [13], [16]; the efficiency advantage reported for MQTT in streaming telemetry [14], [15] matters less here, since the panel exchanges a few hundred bytes per user action.

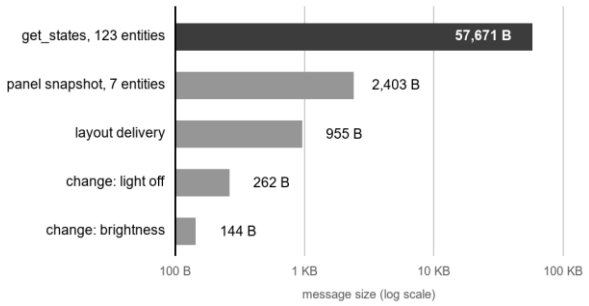


Fig. 5. Measured message sizes on a logarithmic scale. The panel's snapshot is 24 times smaller than the full state dump and grows with the layout, not with the installation.

C. Resource Usage

TABLE IV
Resource Usage of the Panel Firmware

Resource	Used / available
Flash (application partition)	1,198,037 / 3,145,728 B (38.1%)
Static RAM (.data + .bss)	123,024 B; ~1.5 KB below linker limit
LVGL pool, free after first draw	15,652 / 40,960 B
Free heap after start-up (minimum)	161,028 B (158,632 B)
Network task stack, free after auth	8,272 / 12,288 B
Icon fonts (16 glyphs, two sizes)	≈7.6 KB flash, 0 B pool

Table IV lists resource usage of the final firmware. Flash and heap leave ample room; the graphics pool had 15.6 KB free after the first full draw, with a largest free block of 15.0

KB, and the network stack had 8.3 KB spare after authentication. The tightest constraint proved to be static data memory, which is bounded by the linker's data segment rather than by the 320 KB that the build report shows as total RAM: enlarging the graphics pool to 48 KB failed to link, as did adding one further 2 KB buffer. Larger pools should therefore be allocated from the heap at start-up. Replacing per-entity HTTP requests with the subscription also removed the HTTP client library and 34 KB of flash. Because the panel is mains powered, Wi-Fi modem sleep is disabled; low-power operation, studied for the ESP32 in [17], was not a goal.

D. Field Reliability Case Study

During testing the panel intermittently showed no link, tiles read '--', and faded tiles from the disconnected state appeared to linger between pages. The first version fetched initial state with one HTTP request per entity, issued from inside the WebSocket receive callback. Those requests timed out, and with seven entities and a 3 s timeout the callback could block for up to 21 s; the WebSocket heartbeat could not run, so the library closed the connection and the cycle repeated. Moving state delivery onto the subscription removed every request that could block the callback.

The panel then failed to join the network at all, and the new Wi-Fi logging reported reason 201, ‘no access point found.’ The laptop's Wi-Fi adapter has a single radio, so the hotspot must share the channel of the laptop's own Wi-Fi connection. Whenever the router moved that connection to 5 GHz, the hotspot moved with it, regardless of its 2.4 GHz setting, and became invisible to the panel; the router's band steering made this appear random. Pinning the adapter to 2.4 GHz resolved it: the panel joined at −35 dBm, loaded its state on the first connection, and the laptop reached it with no packet loss. A pre-demonstration check script now verifies the setting. Logging that names its causes turned a fault that had persisted for days into one diagnosed within a session.

E. Design Trade-offs

Several choices trade generality for predictability on a device with no memory to spare. Restarting to apply a layout costs a few seconds but avoids fragmenting the graphics pool, where an allocation failure would freeze the screen. A fixed string arena caps layout size but guarantees that a layout can never take memory the interface needs. Filtered, fixed-size parsing rejects information the panel does not use, at the cost of explicit overflow checks. Icons as font glyphs rather than images keep the graphics pool untouched. A desktop tool written against the standard library alone means the companion software installs nothing beyond one WebSocket package. In each case the panel's worst-case behaviour is known in advance, which matters more for a fixture on a wall than peak flexibility.

The case study also shows how a design flaw can amplify an environmental fault. A flaky network alone would have cost the panel a few readings; issuing blocking requests inside the receive callback turned each timeout into a dropped connection. Removing the blocking path made the panel robust to the fault even before its cause was found.

F. Limitations

The connection is unencrypted and the access token is compiled into the firmware, which is acceptable on an isolated local network but would need TLS and provisioning for wider deployment. Layout delivery is not acknowledged to the designer. The WebSocket library rejects frames above 15 KB, so an entity with very large attributes could push a snapshot past that limit. Tiles are placed by order in a fixed grid, without multi-cell widgets. The evaluation used Home Assistant's simulated devices, and latency was measured at the server rather than from touch to physical device.

IV. Conclusion

HomeMesh shows that a low-cost microcontroller without external RAM can serve as a responsive wall panel for every device a Home Assistant server knows about, provided the work is divided so that discovery happens on a PC and the panel handles only small, bounded messages. Delivering the layout as data over the existing connection lets the panel be reconfigured without reflashing, and the measurements show that the subscription keeps traffic proportional to the layout rather than to the installation, with ample memory to spare. The field case study shows the value of firmware that reports the causes of its own failures. Future work includes encrypted connections, multi-cell widgets with inline controls, logging to the board's microSD card, a dedicated access point so that the panel does not depend on a laptop hotspot, and end-to-end latency measurement with physical devices.

Acknowledgment

The authors thank their faculty guide, Suhasini S, for guidance and feedback throughout this project.

References

[1] M. J. C. S. Reis and C. Serôdio, "Edge AI for real-time anomaly detection in smart homes," *Future Internet*, vol. 17, no. 4, Art. no. 179, 2025, doi: 10.3390/fi17040179.

[2] S. N. Swamy, P. Shivanarayanamurthy, G. Purohit, and K. S. Raju, "An edge computing-assisted Internet of Things-based secure smart home," in *Lecture Notes on Data Engineering and Communications Technologies*. Singapore: Springer, 2022, pp. 159–173, doi: 10.1007/978-981-16-7182-1_14.

[3] C. Veigl, B. Klaus, B. Aigner, and M. Wagner, "Universal access panel: A novel approach for accessible smart homes and IoT," in *Lecture Notes in Computer Science*. Cham: Springer, 2022, pp. 148–158, doi: 10.1007/978-3-031-08645-8_18.

[4] D. Russo, V. Miori, G. Tolomei, and D. Belli, "Home automation interoperability: Two decades of lessons learned and future prospects into the development of IoT ecosystems," *Internet of Things*, vol. 37, Art. no. 101906, 2026, doi: 10.1016/j.iot.2026.101906.

[5] S. Madadi-Barough, P. Ruiz-Blanco, J. Lin, R. Vidal, and C. Gomez, "Matter: IoT interoperability for smart homes," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 106–112, 2025, doi: 10.1109/MCOM.001.2400274.

[6] C. Stolojescu-Crisan, C. Crisan, and B.-P. Butunoi, "An IoT-based smart home automation system," *Sensors*, vol. 21, no. 11, Art. no. 3784, 2021, doi: 10.3390/s21113784.

[7] G. Verma, S. Pachauri, A. Kumar, D. Patel, A. Kumar, and A. Pandey, "Smart home automation with smart security system over the cloud," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2023, pp. 1–7, doi: 10.1109/ICCCNT56998.2023.10306548.

[8] R. Tse, B. L. Muhammad-Bello, and I. A. Salihu, "Design and prototype implementation of an ESP32-based web smart home automation system

for remote appliance control," *UMYU Scientifica*, vol. 5, no. 3, pp. 44–58, 2026, doi: 10.56919/usci.2653.004.

[9] P. Gupta and I. O. Prabha M, "A survey of application layer protocols for Internet of Things," in *Proc. Int. Conf. Commun. Inf. Comput. Technol. (ICCICT)*, 2021, pp. 1–6, doi: 10.1109/ICCICT50803.2021.9510140.

[10] P. K. Donta, S. N. Srirama, T. Amgoth, and C. S. R. Annavarapu, "Survey on recent advances in IoT application layer protocols and machine learning scope for research directions," *Digit. Commun. Netw.*, vol. 8, no. 5, pp. 727–744, 2022, doi: 10.1016/j.dcan.2021.10.004.

[11] C. Bayılmış, M. A. Ebleme, Ü. Çavuşoğlu, K. Küçük, and A. Sevin, "A survey on communication protocols and performance evaluations for Internet of Things," *Digit. Commun. Netw.*, vol. 8, no. 6, pp. 1094–1104, 2022, doi: 10.1016/j.dcan.2022.03.013.

[12] Lalhriatpuii, Ruchi, and V. Wasson, "Comprehensive exploration of IoT communication protocol: CoAP, MQTT, HTTP, LoRaWAN and AMQP," in *Communications in Computer and Information Science*. Cham: Springer, 2024, pp. 261–274, doi: 10.1007/978-3-031-75861-4_23.

[13] M. Mishra and S. R. N. Reddy, "Practical assessment of application layer protocols for resource constraint IoT applications," in *Smart Innovation, Systems and Technologies*. Singapore: Springer, 2021, pp. 549–559, doi: 10.1007/978-981-16-0878-0_54.

[14] S. Kaur and V. Khanna, "Implementation and comparison of MQTT, WebSocket, and HTTP protocols for smart room IoT application in Node-RED," in *Internet of Things*. Cham: Springer, 2022, pp. 165–193, doi: 10.1007/978-3-030-89554-9_8.

[15] O. Ajayi, A. Bagula, J. Bode, and M. Damon, "A comparison of publish-subscribe and client-server models for streaming IoT telemetry data," in *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*. Cham: Springer, 2023, pp. 129–139, doi: 10.1007/978-3-031-35883-8_9.

[16] M. I. Yamin, A. Ariman, R. M. Dinata, and N. Purnomo, "Comparative performance evaluation of MQTT, WebSocket, and Firebase on ESP32-C3 for real-time IoT communication," *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 10, no. 3, pp. 3917–3923, 2026, doi: 10.36040/jati.v10i3.18056.

[17] H. Demir, I. Selcuk, and A. E. Cosgun, "Power consumption analysis of ESP32 and Arduino Nano for low-power applications," in *Proc. 9th Int. Symp. Innov. Approaches Smart Technol. (ISAS)*, 2025, pp. 1–4, doi: 10.1109/ISAS66241.2025.11101965.